

JALON 5 — Chapitre X : Politique de Tests & Automatisation

Projet : CALENDRIA
Auteur : BOUGHERARA Safi
Date : 31 Mai 2026

1. Stratégie Générale de Tests

La qualité de l'application Calendria est assurée par une approche de test à plusieurs niveaux, combinant des tests automatisés côté backend, des tests de compilation côté frontend, et une validation manuelle structurée pour les interfaces interactives (plan de salle et chatbot).

2. Tests Unitaires Backend (PHPUnit)

Nous avons mis en place le framework de test standard **PHPUnit** pour tester la logique métier du serveur de manière isolée.

a) Périmètre Couvert

Les tests unitaires ciblent en priorité les composants cruciaux qui portent la logique métier de l'application :

- DisponibiliteService** : Validation du moteur de calcul d'occupation, détection des conflits de réservations (marge de repas + buffer de nettoyage), et calcul de créneaux alternatifs.
- ChatbotService** : Simulation des flux conversationnels avec mock de la réponse Gemini.

b) Exemple de Cas de Test Unitaire (`DisponibiliteServiceTest.php`)

Nous testons les scénarios de limite pour garantir la stabilité de notre algorithme d'attribution de table. Par exemple, le test ci-dessous valide le comportement du service lorsque l'identifiant du restaurant fourni est invalide (doit renvoyer `disponible => false`) :

```
<?php

namespace App\Tests\Service;

use App\Entity\Restaurant;
use App\Entity\Table;
use App\Repository\ReservationRepository;
use App\Repository\RestaurantRepository;
use App\Service\DisponibiliteService;
use Doctrine\ORM\EntityManagerInterface;
use PHPUnit\Framework\TestCase;

class DisponibiliteServiceTest extends TestCase
```

```

{
    public function testVerifierDisponibiliteRestaurantNonTrouve()
    {
        // 1. Instanciation des mocks de dépendances de Doctrine
        $entityManager = $this->createMock(EntityManagerInterface::class);
        $reservationRepository = $this->createMock(ReservationRepository::class);
        $restaurantRepository = $this->createMock(RestaurantRepository::class);

        // 2. Configuration du comportement attendu du repository
        $restaurantRepository->expects($this->once())
            ->method('find')
            ->with(999)
            ->willReturn(null);

        // 3. Exécution du service et assertions
        $service = new DisponibiliteService($entityManager, $reservationRepository, $restaurantRepository);
        $result = $service->verifierDisponibilite(999, '2026-06-01', '12:00', 2);

        $this->assertFalse($result['disponible']);
        $this->assertNull($result['table']);
        $this->assertEmpty($result['alternatives']);
    }
}

```

3. Tests de Compilation Frontend & Type-Checking

Pour la partie frontend (React + TypeScript + Vite) :

- **Type-checking strict** : Nous utilisons le compilateur TypeScript (`tsc -b`) pour vérifier à chaque compilation la cohérence de nos typages, des interfaces de données de nos services (`restaurant.service.ts` , `reservation.service.ts`) et de nos composants visuels.
- **Build de production** : La commande `npm run build` est intégrée à notre workflow de développement pour valider la génération des bundles statiques finaux sans aucune erreur ni avertissement bloquant.

4. Vérification Manuelle & Validation Fonctionnelle

En complément des tests de code, des protocoles de tests manuels rigoureux ont été suivis pour valider l'expérience utilisateur et l'interaction des canaux externes :

a) Validation du Plan de Salle Interactif (Timeline)

- **Scénario de test** : Sélectionner la date d'une réservation factice (ex: `2026-06-01`). Faire glisser le curseur (MUI Slider) sur le créneau de la réservation (ex: `12h30`).
- **Comportement attendu** : La table correspondante (ex: Table 5) doit immédiatement passer au rouge (Occupée) et afficher les détails du client concerné au survol de la souris (Tooltip). Les minutes précédant le créneau doivent colorer la table en orange (Arrivée Imminente).

b) Validation du Chatbot IA Conversationnel (Telegram & Web Widget)

- **Scénario de test** : Ouvrir la conversation avec le bot et envoyer : « *Je voudrais réserver pour demain à 13h00 pour 4 personnes.* »
 - **Comportement attendu** :
 1. Le bot vérifie l'existence d'une table disponible de capacité ≥ 4 à cet horaire.
 2. Si disponible, il demande le nom et le numéro de téléphone pour compléter les exigences de réservation.
 3. Une fois validé, le bot confirme la réservation et celle-ci s'affiche instantanément sur l'administration (avec synchronisation de la couleur de la table correspondante).
 4. Si la capacité maximale est dépassée (ex: 12 personnes pour une capacité max de 6), le bot intercepte immédiatement la demande et propose un contact téléphonique direct.
-

5. Outils Employés

1. **PHPUnit** : Exécution des tests unitaires backend.
 2. **TypeScript** (`tsc`) : Analyse de code statique et validation des contrats d'interface React.
 3. **Vite** : Packaging et validation de la structure du bundle frontend.
 4. **Localtunnel** : Tunneling sécurisé pour exposer localement le webhook du bot Telegram et valider les webhooks de manière réaliste en phase de développement.
-

6. Résultats Actuels & Plan d'Action

- **Statut des tests unitaires** : Tous les tests unitaires configurés s'exécutent avec succès.
- **Statut de la compilation front** : La commande `npm run build` produit un bundle optimisé en 2.12 secondes sans erreurs.
- **Plan d'Action** : Intégrer de nouveaux cas de tests unitaires pour couvrir les calculs complexes d'horaires d'ouverture des services de restaurants lors du Jalon 6, et configurer la pipeline de tests automatiques dans GitHub Actions lors de la mise en production.